



Sistemas Operativos

File System I

Introducción

Conceptos clave:

- Propiedades fundamentales
- Inodos
- Dentries
- Directorios
- Open Files



¿Qué es un File System?

Un **file system** o **sistema de archivos** permite a los usuarios organizar sus datos para que se persistan a través de un largo período de tiempo.

Formalmente un file system es:

> Una abstracción del sistema operativo que provee datos persistentes con un nombre.

Datos persistentes son aquellos que se almacenan hasta que son explícitamente (o accidentalmente 🙄) borrados, incluso si la computadora tiene un desperfecto con la alimentación eléctrica.



¿Qué es un File System?

El hecho de que los datos tengan un nombre es con la intención de que un ser humano pueda acceder a ellos por un identificador que el sistema de archivos le asocia al archivo en cuestión.

Además, esta posibilidad de identificación permite que una vez que un programa termina de generar un archivo otro lo pueda utilizar, permitiendo así compartir la información entre los mismos.



Decisiones de diseño

El sistema de archivos de Unix tomó algunas decisiones de diseño fundamentales que influenciaron los sistemas de archivos posteriores:

- Una estructura jerárquica
- Tratamiento consistente de los datos de los archivos
- La habilidad de crear y borrar archivos
- Crecimiento dinámico de archivos
- Protección de los datos del archivo
- El tratamiento de los periféricos como archivos

Estructura jerárquica

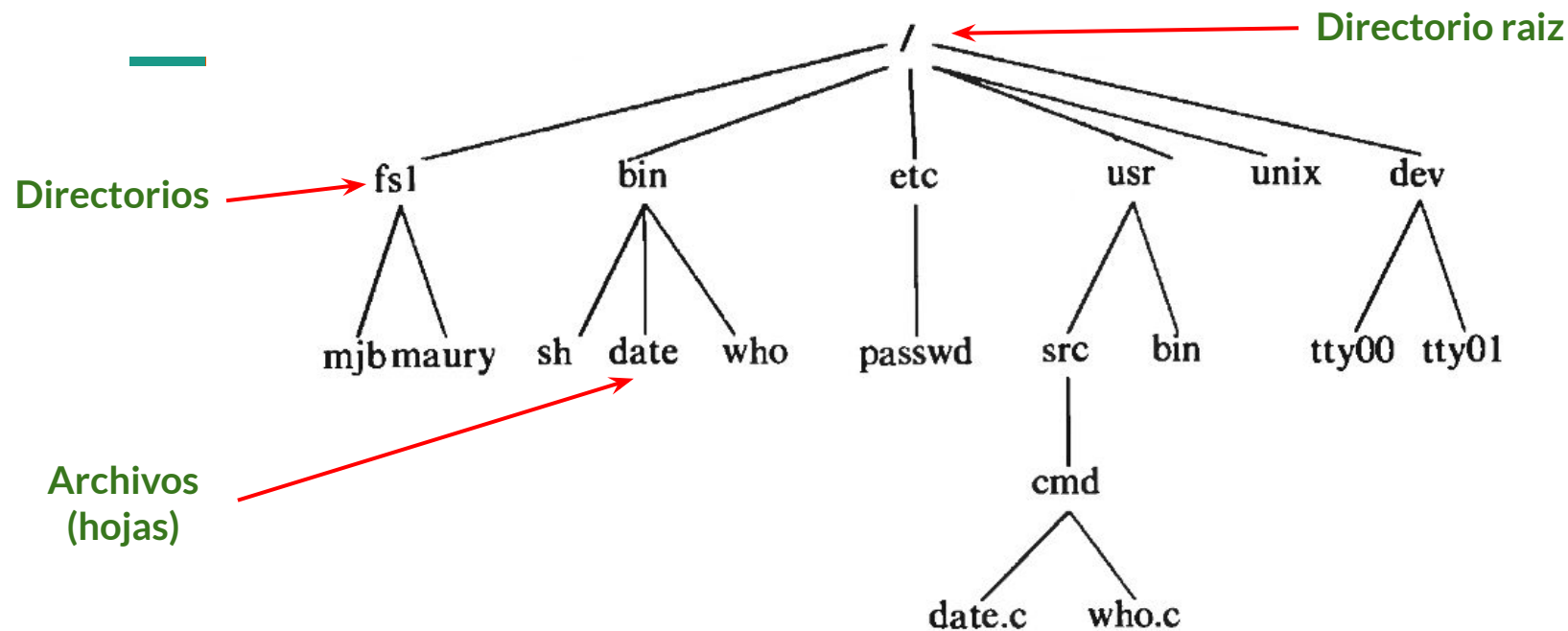


Figure 1.2. Sample File System Tree



Tratamiento consistente de archivos

Los programas ven a los archivos como streams (flujos) de bytes.

Al nivel de las system calls (eg. read y write) no se asume que haya una codificación o formato (eg. ASCII, Unicode, etc.)

Todos los archivos son una secuencia de bytes, y nada mas



Permisos

Los archivos tienen asociados permisos.

El esquema de permisos clásico de UNIX:

- 3 permisos: read, write y execute
- 3 clases de usuarios: owner, grupo, todos los demas

Dispositivos como archivos



Todo es un archivo en UNIX (eg. discos rigidos, CD-Rom, terminales, el proc filesystem).

Esto permite un acceso uniforme a los datos de los dispositivos, porque todos los dispositivos son archivos, y los archivos son secuencias de bytes!

Esto permite un manejo de permisos unificado, porque todos los dispositivos son archivos y los archivos tienen permisos!

Ejemplo:

Para clonar /dev/sdc (250G) a /dev/sdd (250G) en Linux:

```
# dd if=/dev/sdc of=/dev/sdd bs=64K conv=noerror,sync
```

Archivos



Un archivo es una colección de datos con un nombre específico.

Por ejemplo `/home/mariano/MisDatos.txt`.

Los archivos proveen una abstracción de más alto nivel que la que subyace en el dispositivo de almacenamiento; un archivo proporciona un nombre único y con significado para referirse a una cantidad arbitraria de datos.

Por ejemplo, `/home/mariano/MisDatos.txt` podría estar guardada en el disco en los **bloques** `0x0A23D42F`, `0xE3A2540F` y en `0x5567Ae34`; es evidente que es muchísimo más fácil recordar `/home/mariano/MisDatos.txt` que esa lista de bloques en los que se almacena el archivo los datos.

Archivos



Metadata: información acerca del archivo que es comprendida por el Sistema Operativo, esta información es :

- tamaño

- fecha de modificación

- propietario

- información de seguridad (que se puede hacer con el archivo.

Datos: son los datos propiamente dichos que quieren ser almacenados. Desde el punto de vista del Sistema Operativo, un archivo o file no es más que un arreglo de bytes sin tipo.

Archivos



Para poder ver la metadata de un archivo existen varias opciones:

```
$ ls -lisan prueba  
28332376 4 -rw-rw-r-- 1 1000 1000 1457 ene 13 20:48 prueba
```



Archivos

o de una forma más human-readable:

```
$ stat prueba
Fichero: 'prueba'
Tamaño: 1457      Bloques: 8      Bloque E/S: 4096      fichero regular
Dispositivo: 803h/2051d      Nodo-i: 28332376      Enlaces: 1
Acceso: (0664/-rw-rw-r--)  Uid: ( 1000/ mariano)  Gid: ( 1000/ mariano)
Acceso: 2017-05-31 15:17:23.460862535 -0300
Modificación: 2017-01-13 20:48:39.716785878 -0300
      Cambio: 2017-01-13 20:48:39.760786398 -0300
Creación: -
```

Archivos



Un archivo se implementa como un dentry apuntando a un inodo.

Que cosa?...

Inodo

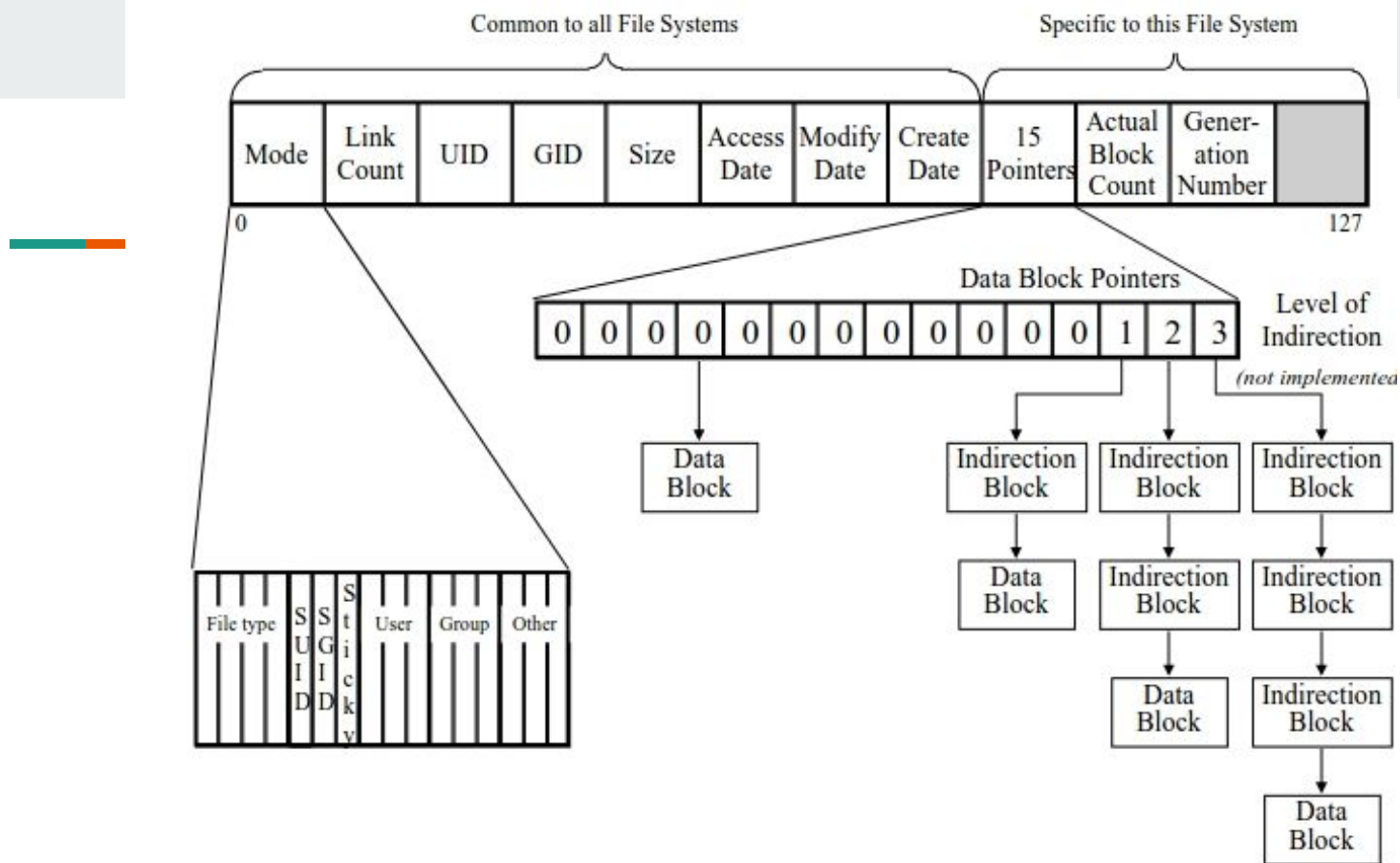


Un inodo (index node) almacena información sobre un archivo.

Un inodo contiene metadatos del archivo, pero **no el contenido del archivo en sí**.

Incluye detalles como:

- Tamaño del archivo
- ID del propietario y del grupo
- Permisos del archivo (lectura, escritura, ejecución)
- Tiempos de acceso, modificación y cambio del inodo
- Número de enlaces (enlaces duros) al archivo
- Punteros a los bloques de disco donde se almacena el contenido real del archivo



El inodo contiene punteros a bloques de datos que contienen los datos del archivo en si

- El inodo NO contiene los datos del archivo
- El inodo SI contiene la metadata del archivo

Tipos de Inodo en Linux



Los tipos de inodo en Linux nos dan idea de la variedad de “objetos” que podemos anclar al filesystem

Se especifican en el campo **mode**:

Macro	Value	Description
S_IFSOCK	0140000	Socket file
S_IFLNK	0120000	Symbolic link
S_IFREG	0100000	Regular file
S_IFBLK	0060000	Block device file
S_IFDIR	0040000	Directory
S_IFCHR	0020000	Character device file
S_IFIFO	0010000	FIFO (named pipe)

Dentry



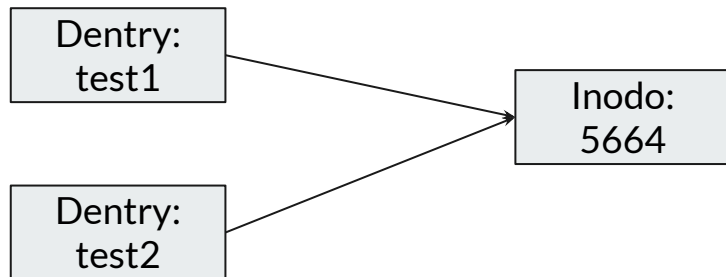
Un Dentry (directory entry) representa la relación entre el nombre de un archivo y su inodo correspondiente.

Los directorios son listas de dentries.

La función principal de un dentry es ayudar a resolver nombres de archivos y rutas en el sistema de archivos. Cuando el sistema de archivos busca un archivo, utiliza las dentries para traducir la ruta al inodo correspondiente, que contiene la información del archivo.

Dentry

Puede haber más de un Dentry apuntando a un mismo inodo. Esto es efectivamente un hard link



Los inodos no estan asociados a un sistema jerarquico. Los dentries al formar directorios

Directorios



Los directorios son archivos especiales de tipo “directorio”. El contenido de estos archivos es una lista de dentries.

- Un dentry no apunta a otro dentry, solo a inodos
- Un inodo no apunta a otros inodos, solo a datos.
- Pero un dentry puede apuntar a un inodo tipo directorio, que apunta a datos y los datos contienen una lista de dentries. Así se implementa el sistema jerárquico!

Directorios

	inode	rec_len	name_len	file_type	name								
0	21	12	1	2	.	\0	\0	\0					
12	22	12	2	2	.	.	\0	\0					
24	53	16	5	2	h	o	m	e	1	\0	\0	\0	
40	67	28	3	2	u	s	r	\0					
52	0	16	7	1	o	l	d	f	i	l	e	\0	
68	34	12	4	2	s	b	i	n					

A tener en cuenta que un directorio es tratado como un archivo normal, no hay un objeto específico para directorios. En unix los directorios son archivos normales que listan los los archivos contenidos en ellos. :: /home/darthmendez/hola.txt

Recapitulando: abstracciones del FS



Abstracción	Entidad del Kernel
Los archivos en si (regulares, directorios, dispositivos, o especiales)	Inodo
Los nombres asociados a los archivos	Dentry

Recapitulando: abstracciones del FS



- Un inodo apunta a datos
- Un dentry apunta a inodos

Pero:

- Un inodo nunca apunta a otros inodos
- Un dentry nunca apunta a otros dentries

La jerarquía del filesystem se forma porque un directorio contiene algunos dentries que apuntan a inodos de tipo `DIRECTORY`, y estos apuntan a bloques de datos que contienen listas de dentries.

Archivos

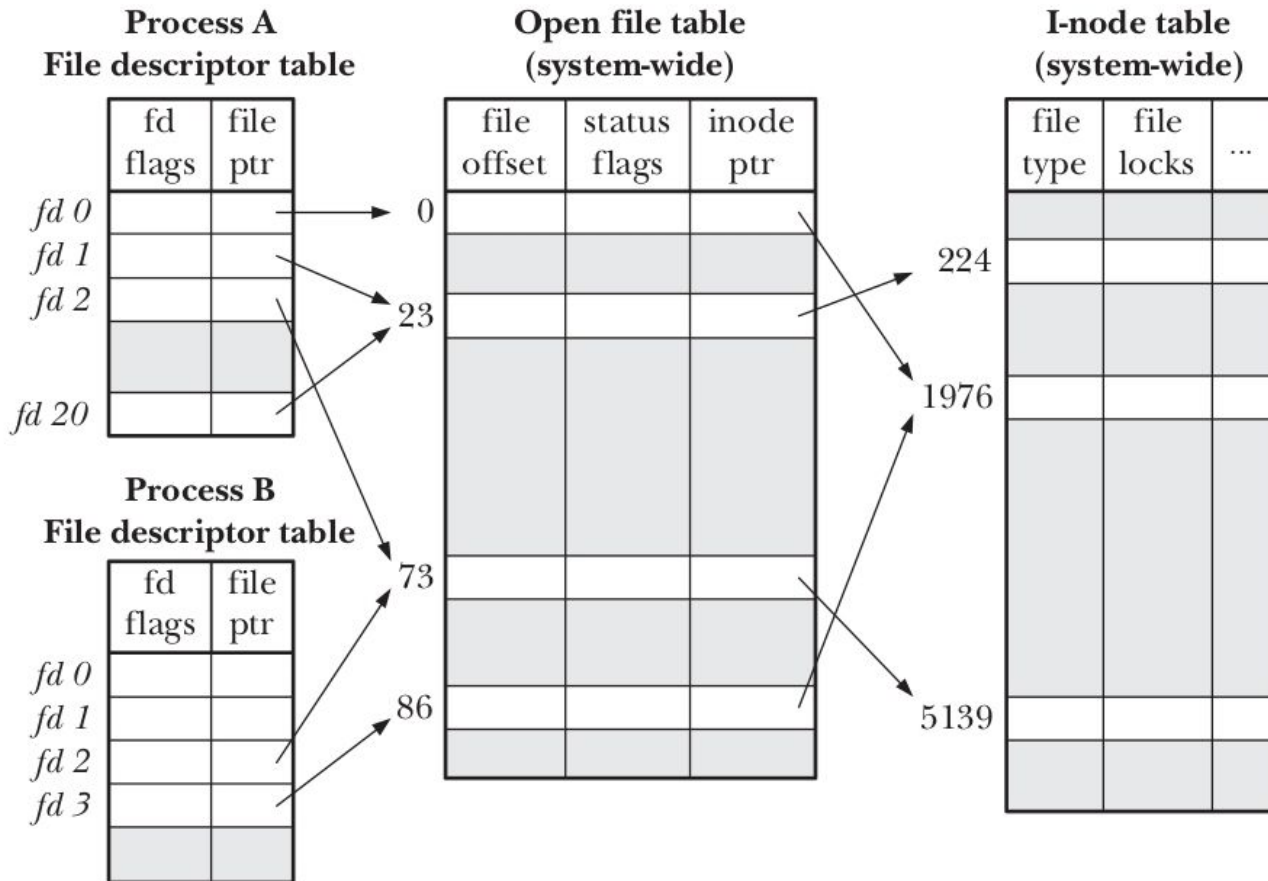


El componente que falta para unir el filesystem con los procesos es la tabla de archivos abiertos

Esta tabla vincula un file descriptor (open, read, write, etc.) con el inodo que representa el archivo.

El file (miembro de la open file table) contiene el offset. Esto es el “cursor” que indica donde se va a escribir o leer a continuacion

Open files



Virtual FileSystem

Conceptos clave:

- Capas de abstracción
- Componentes del VFS



El Virtual File System

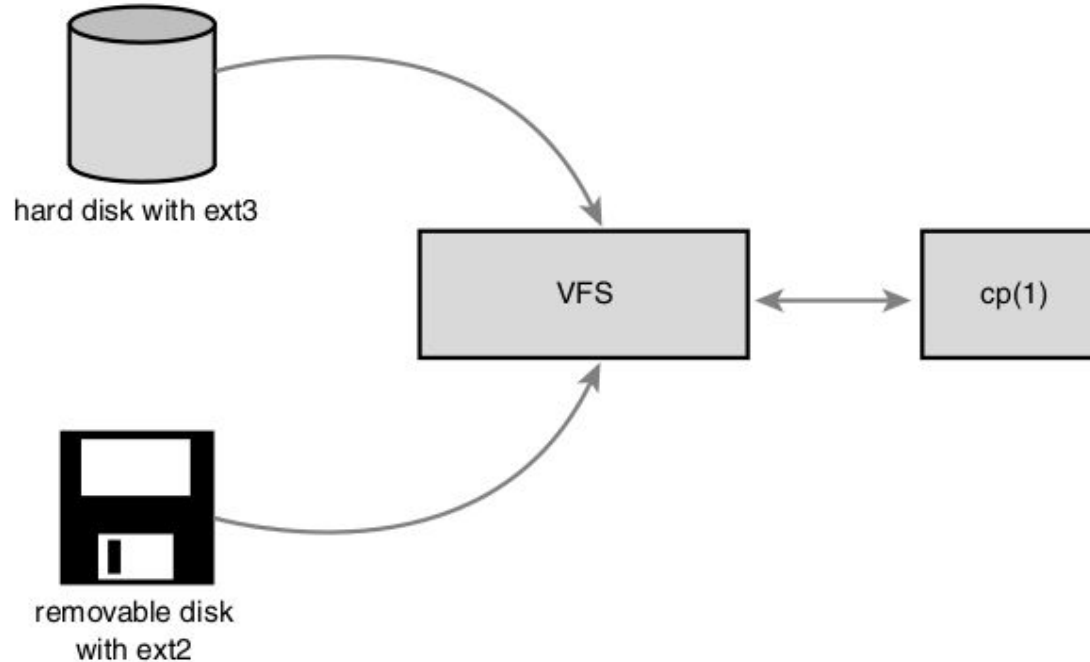
El **Virtual Files System (VFS)** es el subsistema del kernel que implementa la interfaz que tiene que ver con los archivos y el sistema de archivos provistos a los programas corriendo en modo usuario. Todos los sistemas de archivos deben basarse en VFS para :

- coexistir

- inter-operar

Esto habilita a los programas a utilizar las system calls de unix para leer y escribir en diferentes sistemas de archivos y diferentes medios.

El Virtual File System



VFS es el pegamento que habilita a las system calls como por ejemplo `open()`, `read()` y `write()` a funcionar sin que estas necesitan tener en cuenta el hardware subyacente.



FileSystem Abstraction Layer

- Es un tipo genérico de interfaz para cualquier tipo de filesystem que es posible sólo porque el kernel implementa una capa de abstracción que rodea esta interfaz para con el sistema de archivo de bajo nivel.
- Esta capa de abstracción habilita a Linux a soportar sistemas de archivos diferentes, incluso si estos difieren en características y comportamiento.
- Esto es posible porque VFS provee un modelo común de archivos que pueda representar cualquier característica y comportamiento general de cualquier sistema de archivos.

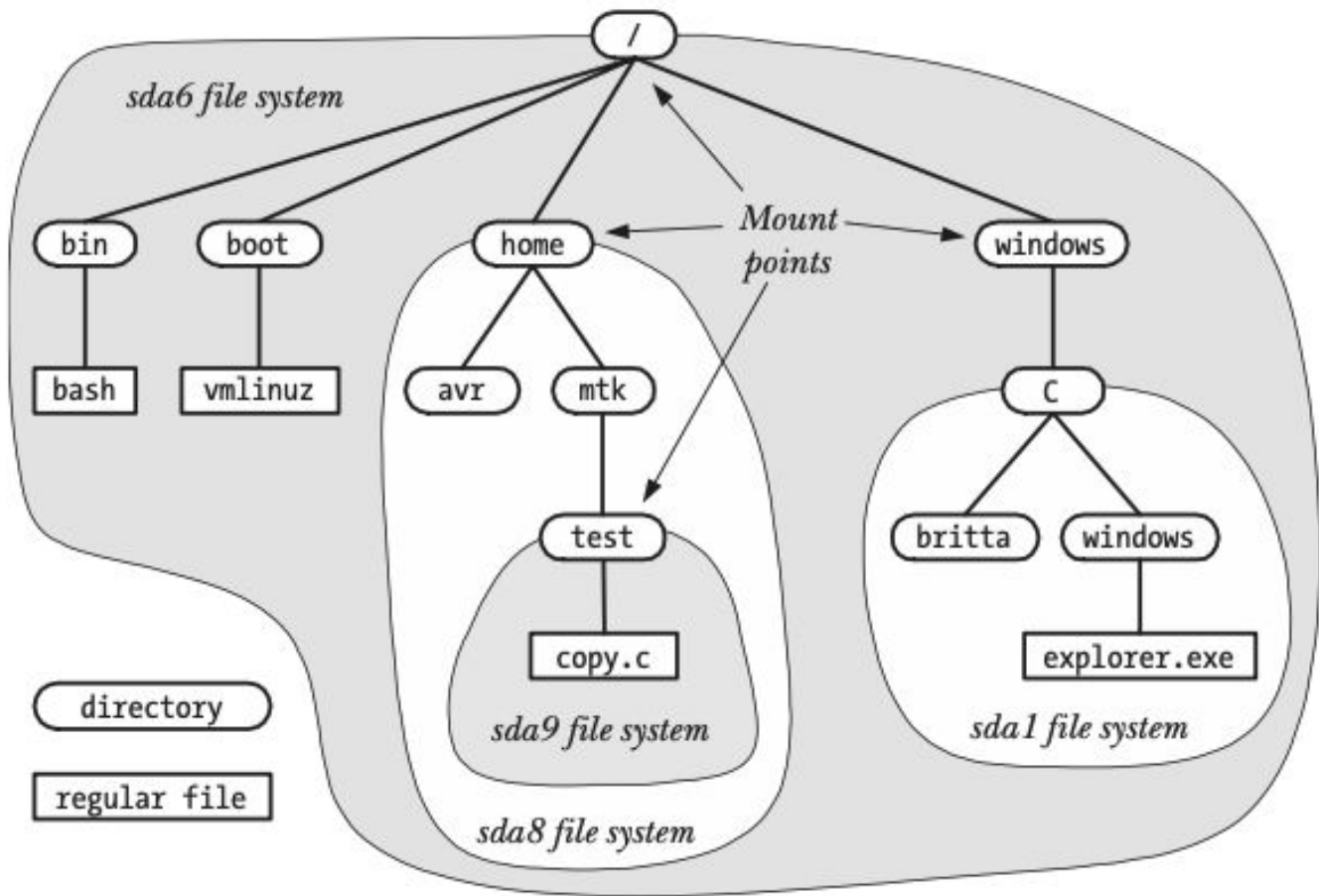
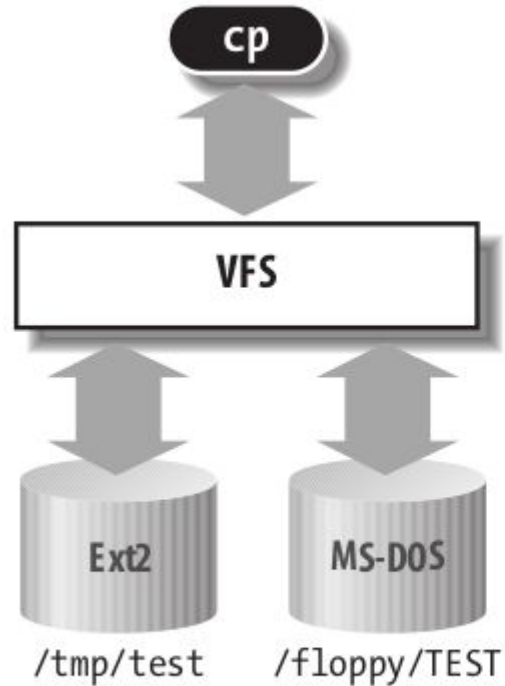


Figure 14-4: Example directory hierarchy showing file-system mount points



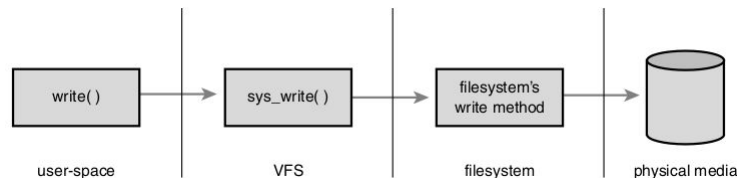
(a)

```
inf = open("/floppy/TEST", O_RDONLY, 0);
outf = open("/tmp/test",
            O_WRONLY|O_CREAT|O_TRUNC, 0600);
do {
    i = read(inf, buf, 4096);
    write(outf, buf, i);
} while (i);
close(outf);
close(inf);
```

(b)

FileSystem Abstraction Layer

FileSystem Abstraction Layer

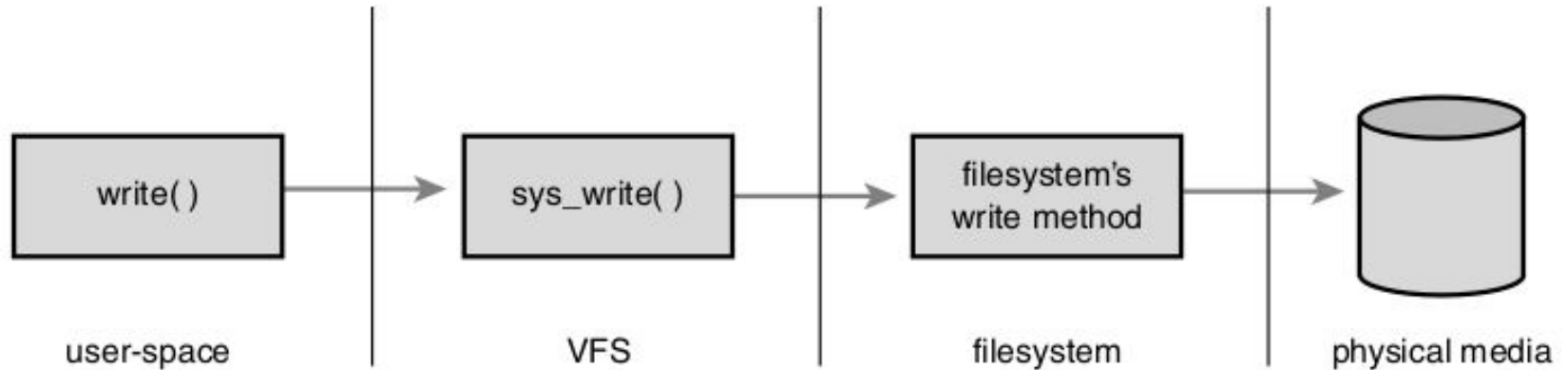


Esta capa de abstracción trabaja mediante la definición de interfaces conceptualmente básicas y de estructuras que cualquier sistema de archivos soporta.

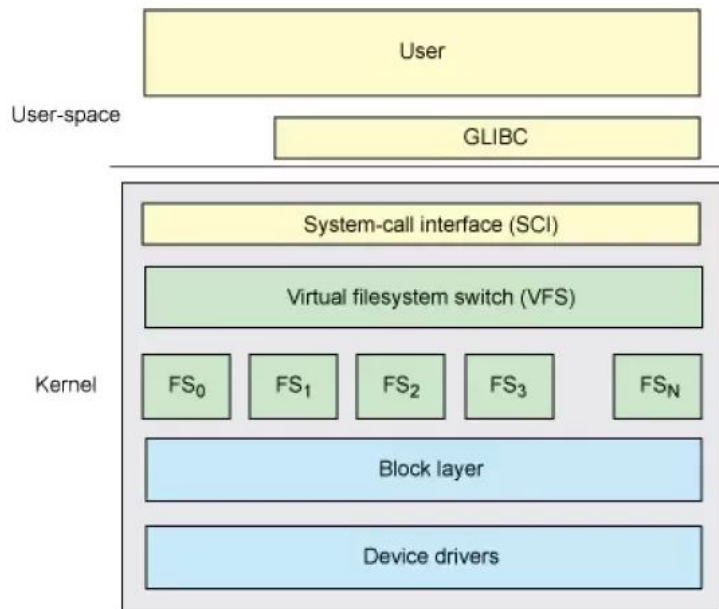
Los filesystems amoldan su visión de conceptos como “esta es la forma de cómo abro un archivo” para matchear las expectativas del VFS, todos estos sistemas de archivos soportan nociones tales como archivos, directorios y además todos soportan un conjunto de operaciones básicas sobre estos.

El resultado es una capa de abstracción general que le permite al kernel manejar muchos tipos de sistemas de archivos de forma fácil y limpia.

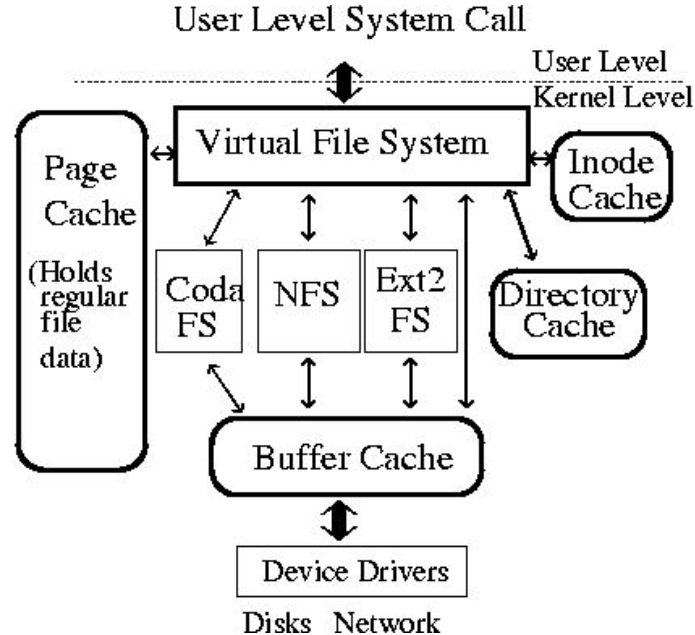
FileSystem Abstraction Layer



FileSystem Abstraction Layer



FileSystem Abstraction Layer



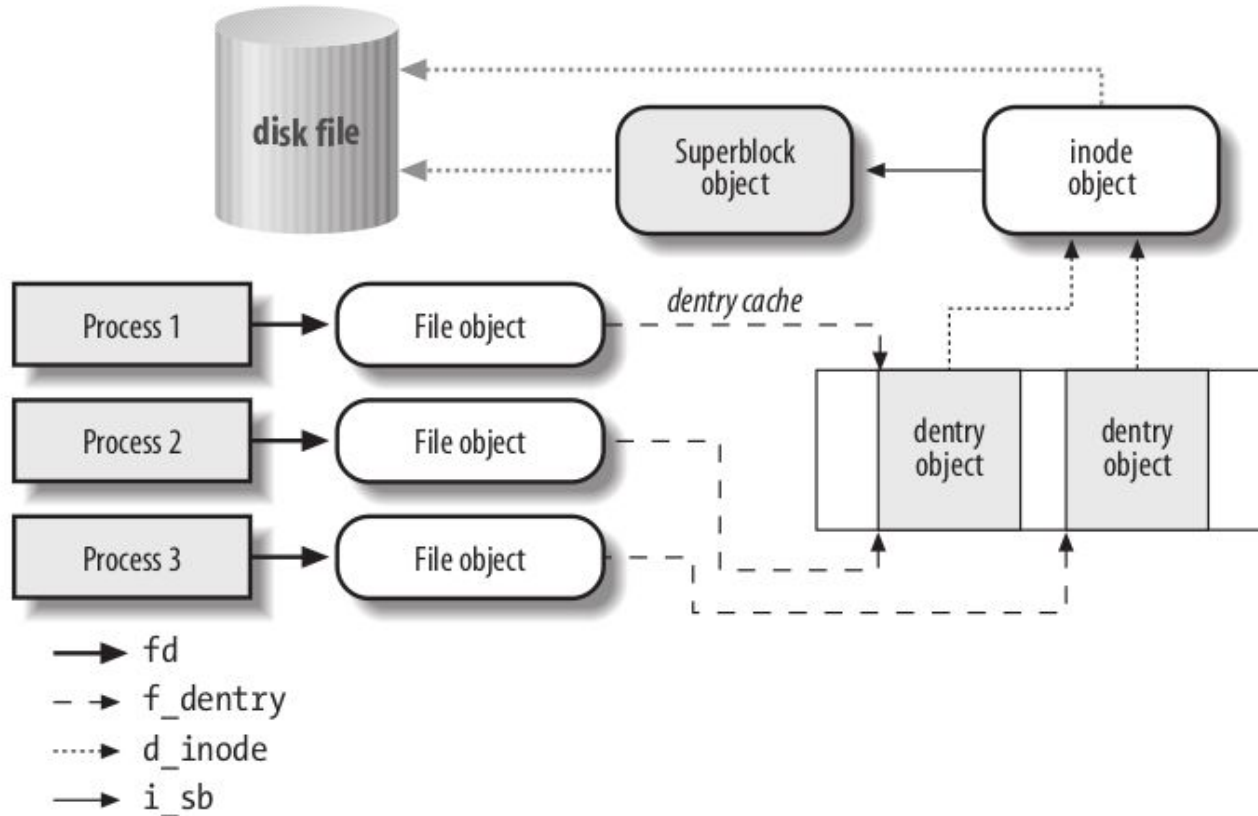
VFS: Sus Objetos



VFS presenta una serie de estructuras que modelan un filesystem, estas estructuras se denominan objetos (programadas en C). Estos Objetos son:

- El **super bloque**, que representa a un sistema de archivos.
- El **inodo**, que representa a un determinado archivo.
- El **dentry**, que representa una entrada de directorio, que es un componente simple de un path.
- El **file** que representa a un archivo asociado a un determinado proceso

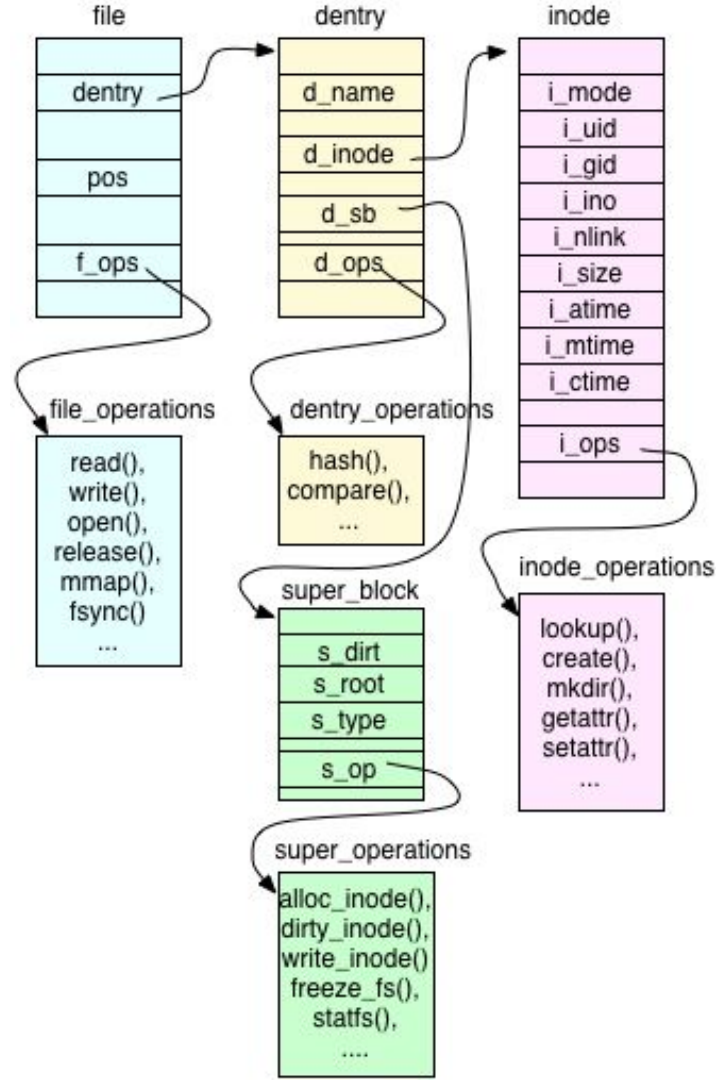
VFS: Sus Objetos



VFS: Operaciones



- Las `super_operations` métodos aplica el kernel sobre un determinado sistema de archivos, por ejemplo `write_inode()` o `sync_fs()`.
- Las `inode_operations` métodos que aplica el kernel sobre un archivo determinado, por ejemplo `create()` o `link()`.
- Las `dentry_operations` métodos que se aplican directamente por el kernel a una determinada directory entry, como por ejemplo, `d_compare()` y `d_delete()`.
- Las `file_operations` métodos que el kernel aplica directamente sobre un archivo abierto por un proceso, `read()` y `write()`, por ejemplo.



VFS: Operaciones

Dispositivos

Conceptos clave:

- Drivers
- /dev
- Dispositivos de bloque y de caracter
- mknod y mount
- Sistemas de archivos virtuales

Estructura para el acceso a dispositivos

- Los dispositivos se representan como archivos especiales de tipo DEVICE
- Se usan las mismas system calls que para operar con archivos normales
- El VFS interactúa con una capa de Drivers
- Los Drivers son los que implementan las operaciones de bajo nivel que interactúan con el hardware.

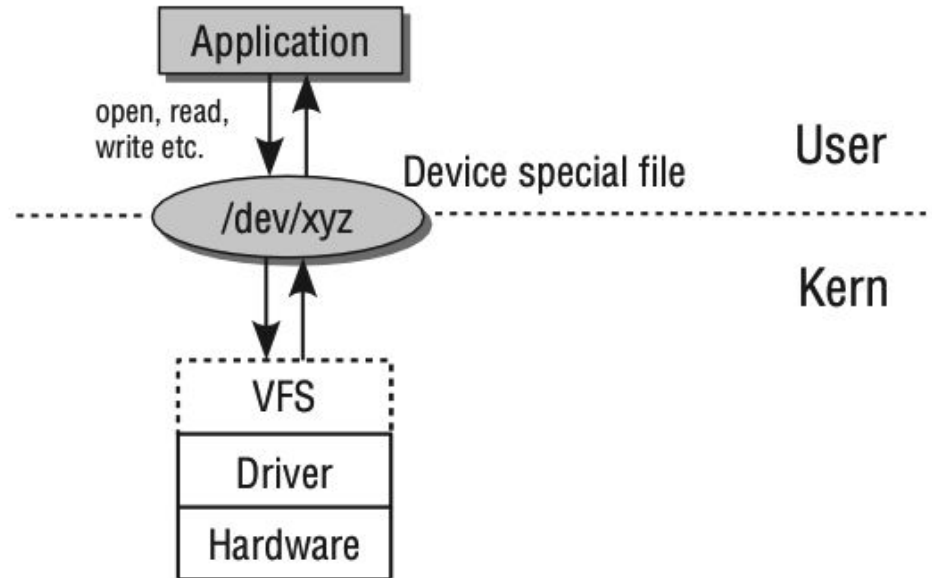


Figure 6-1: Layer model for addressing peripherals.

El directorio `/dev`



En general los dispositivos se cargan como archivos especiales en el directorio `/dev`

Ejemplos de dispositivos que se encuentran ahí (se pueden listar con `ls`):

- `/dev/sda`: El primer disco duro en el sistema.
- `/dev/sda1`: La primera partición del primer disco duro.
- `/dev/ttyS0`: Un puerto serie (como un puerto COM en sistemas Windows).
- `/dev/loop0`: Un dispositivo de bucle que permite tratar un archivo como si fuera un disco.
- `/dev/null`: Un "dispositivo" que descarta cualquier dato que se le escriba.
- `/dev/random`: Proveedor de números aleatorios.



Tipos de dispositivos

Existen dos tipos fundamentales de dispositivos en Unix que se pueden vincular como nodos al filesystem.

- Dispositivos de bloque
- Dispositivos de caracter

Nota: existen también los dispositivos de red, que no se vinculan con el filesystem del sistema

Dispositivos de carácter (character devices)



Los dispositivos de carácter permiten la transferencia de datos byte a byte. Estos dispositivos no almacenan datos en bloques, sino que transmiten la información carácter por carácter, lo que los hace adecuados para dispositivos que no requieren procesamiento de grandes cantidades de datos de una sola vez.

Ejemplos de dispositivos de carácter:

- Terminales (como consolas o dispositivos tty)
- Puertos serie (para dispositivos como ratones o modems)
- Impresoras de tipo línea a línea

En el sistema, estos dispositivos suelen encontrarse en `/dev`, con nombres como `/dev/ttyS0` para un puerto serie.

Dispositivos de bloque (block devices)



Los dispositivos de bloque permiten la transferencia de datos en bloques de tamaño fijo (normalmente de varios bytes). Este tipo de dispositivo es adecuado para hardware que requiere el acceso a grandes cantidades de datos de forma eficiente, como los sistemas de almacenamiento.

Ejemplos de dispositivos de bloque:

- Discos duros
- Unidades de estado sólido (SSD)
- Memorias USB

En el sistema, estos dispositivos también se encuentran en `/dev`, con nombres como `/dev/sda` para un disco duro o una partición específica.

Cómo se vinculan los dispositivos al filesystem



Los dispositivos de I/O tienen un par de **números** asociados llamados números **mayor** y **menor**.

Estos números implementan un sistema muy básico de ruteo hacia los dispositivos en sí.

Major Number. El número mayor **identifica driver** que maneja un grupo específico de dispositivos. El kernel utiliza este número para determinar qué controlador debe gestionar las solicitudes de entrada/salida (I/O) dirigidas a un dispositivo específico.

Minor Number. El número menor **identifica un dispositivo específico dentro del grupo de dispositivos** gestionado por un mismo controlador. En otras palabras, mientras que el número mayor se refiere al controlador, el número menor identifica a un dispositivo particular bajo ese controlador.

Cómo conectan los dispositivos al filesystem



Los dispositivos no son más que archivos especiales de tipo **DEVICE** que se crean usando la system call `mknod`

```
sudo mknod /dev/ttyS1 c 4 65
```

`/dev/ttyS1`: es el nombre del archivo especial que vamos a crear

`C`: indica que este es un dispositivo de caracteres, no bloques (sino sería `b`)

`4`: es el major

`65`: es el minor

En general crear nodos de dispositivo no es necesario, esto lo resuelve el programa **udev** que dinámicamente monitorea y crea nodos de dispositivo. Además resuelve la gestión de major y minors para los cual “se asume” que el administrador del sistema tiene conocimiento de cómo obtenerlos para cada dispositivo

Cómo conectan los dispositivos al filesystem

Mknod crea inodos de tipo Block device o Character device

Macro	Value	Description
S_IFSOCK	0140000	Socket file
S_IFLNK	0120000	Symbolic link
S_IFREG	0100000	Regular file
S_IFBLK	0060000	Block device file 
S_IFDIR	0040000	Directory
S_IFCHR	0020000	Character device file 
S_IFIFO	0010000	FIFO (named pipe)

Cómo conectan los dispositivos al filesystem



Ahora sabemos crear nodos que representan dispositivos, pero el mismo es un archivo es decir bytes. Si es un disco no nos interesan los bloques de bytes, nos interesa el filesystem que esta contenido en los mismos.

```
sudo mount -t ext4 /dev/sda1 /mnt/mi_directorio
```

-t ext4: es el tipo de filesystem que estamos cargando. Esto le permite al kernel saber como interpretar los bloques del dispositivo de bloques.

/dev/sda1: es el dispositivo en bloques origen del filesystem.

/mnt/mi_directorio: es el punto de montaje, es decir, donde se va a colocar el filesystem

Como se monta un filesystem

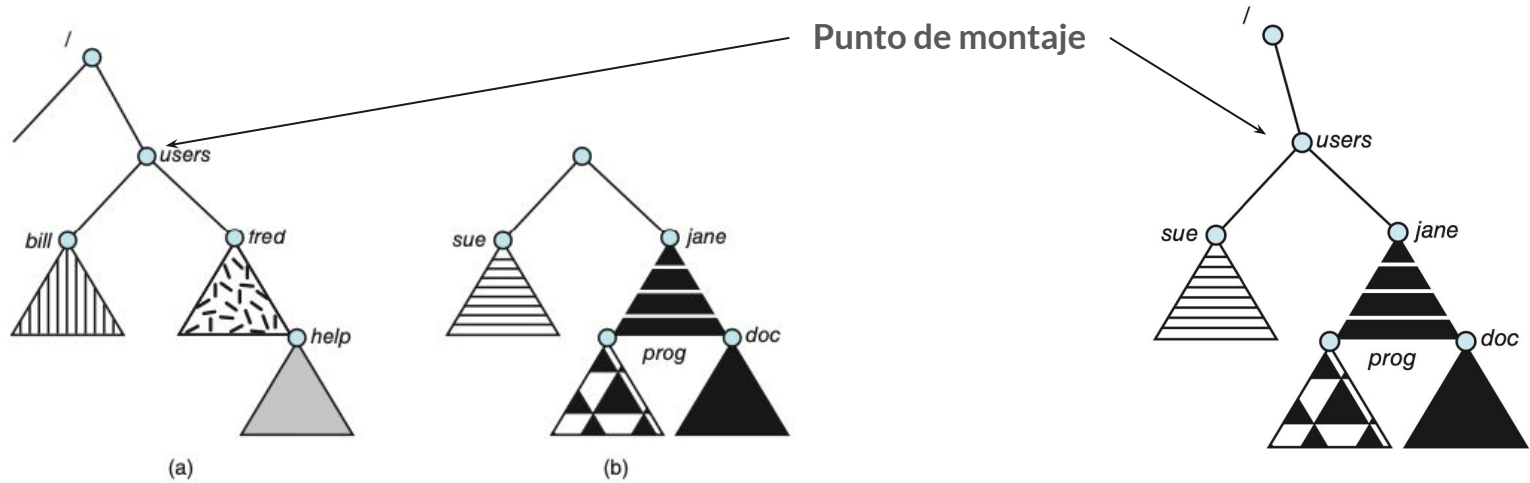


Figure 15.3 File system. (a) Existing system. (b) Unmounted volume.

Figure 15.4 Volume mounted at `/users`.

Filesystems virtuales



No todos los filesystems se corresponden con un dispositivo. Algunos son **enteramente virtuales!**

Ejemplo: procfs

```
mount -t proc proc /proc
```

El procfs (sistema de archivos de procesos) es un sistema de archivos virtual en los sistemas operativos tipo Unix (incluyendo Linux) que proporciona una interfaz para acceder a la información sobre los procesos y otros datos del sistema directamente desde el kernel.

Filesystems virtuales



Que otros tipos de filesystem virtual hay?

Usemos /proc para obtener la respuesta!

```
ubuntu@primary:~$ cat /proc/filesystems
nodev    sysfs
nodev    tmpfs
nodev    proc
nodev    cgroup
          ext4
...
```